



# Rapport — LOG430 Labo 08

## Question 1

**Comment on faisait pour passer d'un état à l'autre dans la saga dans le labo 6, et comment on le fait ici? Est-ce que le contrôle de transition est fait par le même structure dans le code?**

Dans le labo 6, qui est une saga orchestrée, un seul fichier central (`order_saga_controller.py`) contrôle toutes les transitions grâce à une boucle `while/if-elif`. Dans le labo 8, chaque handler publie lui-même le prochain événement Kafka sans l'aide d'un contrôleur central. Le `HandlerRegistry` dispatch les événements au bon handler. La structure de contrôle est donc complètement différente.

**Labo 6 — orchestrateur central (`order_saga_controller.py`) :**

```
while self.current_saga_state is not OrderSagaState.END:
    if self.current_saga_state == OrderSagaState.START:
        self.logger.debug("État initial")
        self.current_saga_state = self.create_order_handler.run()
    elif self.current_saga_state == OrderSagaState.ORDER_CREATED:
        self.decrease_stock_handler = DecreaseStockHandler(self.create_order_handler.order_id)
        self.current_saga_state = self.decrease_stock_handler.run()
    elif self.current_saga_state == OrderSagaState.STOCK_DECREASED:
        self.create_payment_handler = CreatePaymentHandler(self.create_order_handler.order_id)
        self.current_saga_state = self.create_payment_handler.run()
    elif self.current_saga_state == OrderSagaState.STOCK_INCREASED:
        self.delete_order_handler = DeleteOrderHandler(self.create_order_handler.order_id)
        self.current_saga_state = self.delete_order_handler.run()
    elif self.current_saga_state is OrderSagaState.PAYMENT_CREATED or self.current_saga_state is OrderSagaState.STOCK_INCREASED:
        self.logger.debug("Transition à l'état terminal")
        self.current_saga_state = OrderSagaState.END
    else:
        self.is_error_occurred = True
        self.logger.debug(f"L'état de la commande n'est pas valide : {self.current_saga_state}")
        self.current_saga_state = OrderSagaState.END
```

## Labo 8 — chorégraphie décentralisée ( `store_manager.py` + handlers ) :

Enregistrement des handlers dans le `HandlerRegistry` :

```
registry = HandlerRegistry()
registry.register(OrderCreatedHandler())
registry.register(OrderCreationFailedHandler())
registry.register(OrderCancelledHandler())
registry.register(StockDecreasedHandler())
registry.register(StockDecreaseFailedHandler())
registry.register(StockIncreasedHandler())
registry.register(PaymentCreatedHandler())
registry.register(PaymentCreationFailedHandler())
registry.register(SagaCompletedHandler())

consumer_service = OrderEventConsumer(
    bootstrap_servers=config.KAFKA_HOST,
    topic=config.KAFKA_TOPIC,
    group_id=config.KAFKA_GROUP_ID,
    registry=registry
)
```

Chaque handler décide lui-même du prochain événement ( `order_created_handler.py` ) :

```
order_event_producer = OrderEventProducer()
try:
    session = get_sqlalchemy_session()
    check_out_items_from_stock(session, event_data['order_items'])
    session.commit()
    event_data['event'] = "StockDecreased"
except Exception as e:
    session.rollback()
    event_data['event'] = "StockDecreaseFailed"
    event_data['error'] = str(e)
finally:
    session.close()
    order_event_producer.get_instance().send(config.KAFKA_TOPIC, value=event_data)
```

## Question 2

**Sur la relation entre nos Handlers et le patron CQRS : pensez-vous qu'ils utilisent plus souvent les Commands ou les Queries? Est-ce qu'on tient l'état des Queries à jour par rapport aux changements d'état causés par les Commands?**

Les handlers utilisent presque exclusivement des **Commands** : ils modifient le stock, créent des entrées Outbox et mettent à jour les commandes. Ils ne lisent pas de données pour répondre à une requête.

Pour la synchronisation des Queries, c'est partiel. Les commandes sur les orders mettent bien Redis à jour via `modify_order()`. En revanche, `check_out_items_from_stock()` ne met à jour que MySQL, pas le cache Redis du stock.

`modify_order()` dans `write_order.py` — met à jour MySQL et Redis (query side tenu à jour) :

```
order.is_paid = is_paid
order.payment_link = f"http://api-gateway:8080/payments-api/payments/process/{payment_id}"
session.commit()
```

`check_out_items_from_stock()` dans `write_stock.py` — ne touche que MySQL (Redis stock potentiellement désynchronisé) :

```
def check_out_items_from_stock(session, order_items):
    """ Decrease stock quantities in Redis """
    update_stock_mysql(session, order_items, "-")
```

---

## Question 3

**Est-ce qu'une architecture Saga orchestrée pourrait aussi bénéficier de l'utilisation du patron Outbox, ou c'est un bénéfice exclusif de la saga chorégraphiée?**

Oui, une saga orchestrée bénéficierait aussi du patron Outbox. Le problème qu'il résout —

une écriture en base réussie mais le message suivant jamais envoyé suite à un crash — existe dans les deux architectures. Dans le labo 6, si `order_saga_controller.py` crashe après avoir créé la commande mais avant d'appeler `DecreaseStockHandler`, la commande reste incohérente exactement comme dans le labo 8. L'Outbox est un patron de résilience générique, pas lié à la chorégraphie.

La logique de récupération des items non traités au démarrage dans `outbox_processor.py` serait identique dans une saga orchestrée :

```
else:
    self.logger.debug("no item informed")
    session = get_sqlalchemy_session()
    outbox_items = session.query(Outbox).filter(Outbox.payment_id.is_(None)).all()
    for outbox_item in outbox_items:
        event_data = self._get_event_data(outbox_item)
        self._process_outbox_item(event_data, outbox_item)
    if not outbox_items:
        self.logger.info("No outbox items to process.")
    else:
        self.logger.info(f"{len(outbox_items)} outbox items processed.")
    session.close()
```

---

## Question 4

**Qu'est-ce qui arriverait si notre application s'arrête avant la création de l'enregistrement dans la table Outbox?**

Si l'application s'arrête après `check_out_items_from_stock()` dans `OrderCreatedHandler` mais avant le `session.add(new_outbox_item)` dans `StockDecreasedHandler`, le stock est diminué en MySQL mais aucun enregistrement Outbox n'existe. Au redémarrage, `OutboxProcessor().run()` dans `store_manager.py` cherche les items où `payment_id IS NULL` — il n'en trouve pas, donc la commande reste sans `payment_link` pour toujours.

La fenêtre de vulnérabilité se situe entre les deux `session.commit()` séparés dans

stock\_decreased\_handler.py :

```
def handle(self, event_data: Dict[str, Any]) -> None:
    session = get_sqlalchemy_session()
    try:
        new_outbox_item = Outbox(order_id=event_data['order_id'],
                                user_id=event_data['user_id'],
                                total_amount=event_data['total_amount'],
                                order_items=event_data['order_items'])

        session.add(new_outbox_item)
        session.flush()
        session.commit()
        OutboxProcessor().run(new_outbox_item)
    except Exception as e:
        session.rollback()
        self.logger.debug("La création d'une transaction de paiement a échoué : " + str(e))
        event_data['event'] = "PaymentCreationFailed"
        event_data['error'] = str(e)
        OrderEventProducer().get_instance().send(config.KAFKA_TOPIC, value=event_data)
    finally:
        session.close()
```

Pour améliorer l'implémentation, il faudrait envelopper la diminution du stock et la création de l'entrée Outbox dans une seule transaction atomique. Sinon, on pourrait scanner au démarrage les commandes sans `payment_link` dans MySQL et les ajouter à l'Outbox manuellement.